done

CITS 3002
Computer Networks

THE UNIVERSITY OF WESTERN AUSTRALIA

Lab 3: Piggyback and Sliding Windows Protocols

Note: This lab runs for 2 weeks, so don't panic if you don't finish in the first week.

Worth: 2% of the unit

Please note: ask your lab demonstrator about any questions you have regarding previous labs. It will be beneficial for you to have a good foundation early on in the semester, and we are here to help!

In Lecture 3 we introduced the stop-and-wait Data Link Layer protocol, which delivers frames between directly connected hosts. We also recognized that the nature of stop-and-wait results in a lot of time being 'wasted', particularly in the presence of Physical Layer errors. In Labsheet 2 we observed that employing negative-acknowledgments can reduce the waiting time, resulting in an increased data throughput.

Another mechanism to improve stop-and-wait is to piggyback acknowledgments onto data frames. Each frame now carries two sequence numbers: a frame travelling from A to B may include a sequence number describing data being carried from A to B and may include another sequence number acknowledging data previously sent from B to A. The benefit is that fewer frames need to traverse 'the wire', and fewer frames interrupt each network interface-card and operating system.

1. Tasks

TASK 1. PIGGYBACK

The implementation of piggybacking is a modification to stop-and-wait (which we implemented in lab 2):

- When the receiver gets a DLL_DATA frame, it does not immediately send an DLL_ACK frame.
- The receiver waits (for a limited time) until it has its own outgoing DLL_DATA frame to send, and piggybacks the pending DLL_ACK in the outgoing header.
- If no DLL_DATA frame becomes available in a short time, the receiver must send an DLL_ACK frame by itself.

For this exercise, using the network simulator:

1. Add piggybacking to the standard stop-and-wait protocol.
2. Modify the topology file, changing the attributes of message delivery, bandwidth, frame corruption and loss, to determine under which conditions piggybacking offers an advantage.

Please use a 1 second timeout for the piggybacking (1000000usec for start_timer()), and ensure that you call enable_application() for both nodes in reboot_node. Piggybacking won't provide any benefit if there are no data frames being sent by the second node (because there is nothing to piggyback onto).

done

Remember, to perform a valid scientific experiment (which, after all, is what we're doing), we need to hold all conditions/attributes constant and vary just one over time, perform the same experiment for both the stop-and-wait and piggybacking protocols.

Run the following questions with the seed value of 19, and using 90 seconds for the STOPANDWAIT file.

```
python sim.py --execution-duration 90s --silent-nodes --stats-csv
piggyback.csv -S 19 PIGGYBACK
```

As in Labsheet 2, develop a plot to help you to visually compare the results of your experiment. The easiest approach is to modify the plot.py script from Labsheet 2.

TASK 2. SLIDING WINDOW

Sliding-window protocols are Data Link Layer protocols that support multiple outstanding frames between sender and receiver. Again, the goal is to minimize the waiting time required in the standard stop-and-wait protocol. In general, sliding-window protocols have the properties that:

- The sender has a sending window consisting of a sequence (an array) of frames that have been sent but not acknowledged.
- The sender's window size grows as more frames are sent but not yet acknowledged.
- The receiver has a receiving window consisting of frames it is willing to accept. The receiver's window size remains constant.
- Each frame being sent has a sequence number from 0 to 2n-1 (which fits in >n bits). Stop-and-wait has n=1.
- A window size of 1 implies that frames are only accepted in order, and thus an error is detected if an expected frame is missed (a 'later' frame arrives first).

The simplest variant of a sliding-window protocol is termed the go-back-N protocol, in which the receiver simply discards (ignores) all frames after a bad one. This corresponds to a receiver window size of 1.

After it learns of a lost (or corrupted frame), the sender retransmits all unacknowledged frames - it "goes back N frames" and starts again. Notice that in the presence of significant errors that bandwidth is wasted because the receiver only 'buffers' a single frame.

Following the same experimental approach as in the first exercise, implement the go-back-N protocol, and determine under which conditions it is an improvement over the stop-and-wait protocol.

2. Quiz on LMS

To gain 2% allocated mark for this lab, please ensure that you solve lab questions on LMS by the deadline.